

Docker and Kubernetes



Lesson Objectives

Session 1 – Getting started with Docker

Session 2 – Building Docker images

Session 3 – Docker Networking

Session 4 – Docker storage

Session 5 – Making a real project with Docker and NodeJs

Session 6 – Docker Swarm

Session 7-10 – Kubernetes

Session 11 – Final test

Session 4

Docker storage

- Assignment Review & Guides
- Storage overview
- Docker volumes
- Bind mounts
- Store data within containers

By default all files created inside a container are stored on a **writable container layer**.

This means that:

- ❖ The data doesn't persist when that container no longer exists
- ❖ A container's writable layer is tightly coupled to the host machine where the container is running
- ❖ Writing into a container's writable layer requires a storage driver to manage the filesystem. This extra abstraction reduces performance as compared to using ***data volumes***

By default all files created inside a container are stored on a **writable container layer**.

This means that:

- ❖ The data doesn't persist when that container no longer exists
- ❖ A container's writable layer is tightly coupled to the host machine where the container is running
- ❖ Writing into a container's writable layer requires a storage driver to manage the filesystem. This extra abstraction reduces performance as compared to using *data volumes*

Choose the right type of mount

- ❖ **Volumes** are stored in a part of the host filesystem which is managed by Docker (/var/lib/docker/volumes/ on Linux).
- ❖ **Bind mounts** may be stored anywhere on the host system. They may even be important system files or directories.
- ❖ **tmpfs** mounts are stored in the host system's memory only, and are never written to the host system's filesystem.

- ❖ **Volumes** mount a directory on the host into the container at a specific location
- ❖ Can be used to share (and persist) data between containers
- ❖ Directory persists after the container is deleted (Unless you explicitly delete it)
- ❖ Can be created in a Dockerfile or via CLI

Volumes are the preferred mechanism for persisting data generated by and used by Docker containers. While bind mounts are dependent on the directory structure and OS of the host machine, volumes are completely managed by Docker. Volumes have several advantages over bind mounts

Create and manage volumes

Create a volume:

```
$ docker volume create my-vol
```

List volumes:

```
$ docker volume
```

```
Local          my-vol
```

Inspect a volume:

```
$ docker volume inspect my-vol
```

Remove a volume:

```
$ docker volume rm my-vol
```

Start a container with a volume

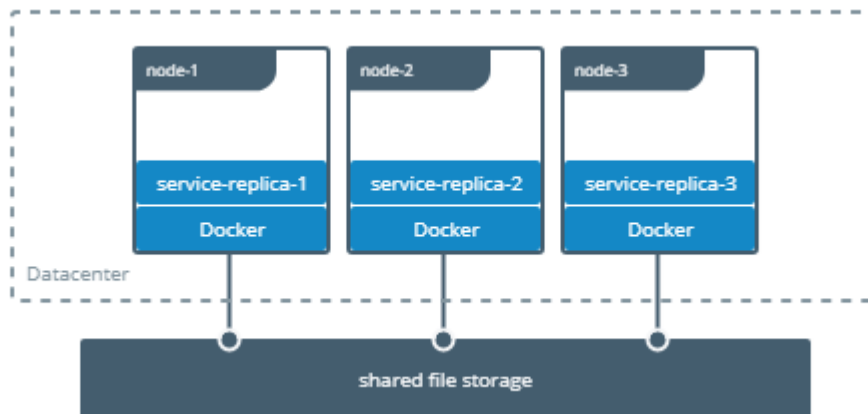
If you start a container with a volume that does not yet exist, Docker creates the volume for you

```
$ docker run -d \  
  --name devtest \  
  -v myvol2:/app \  
  nginx:latest
```

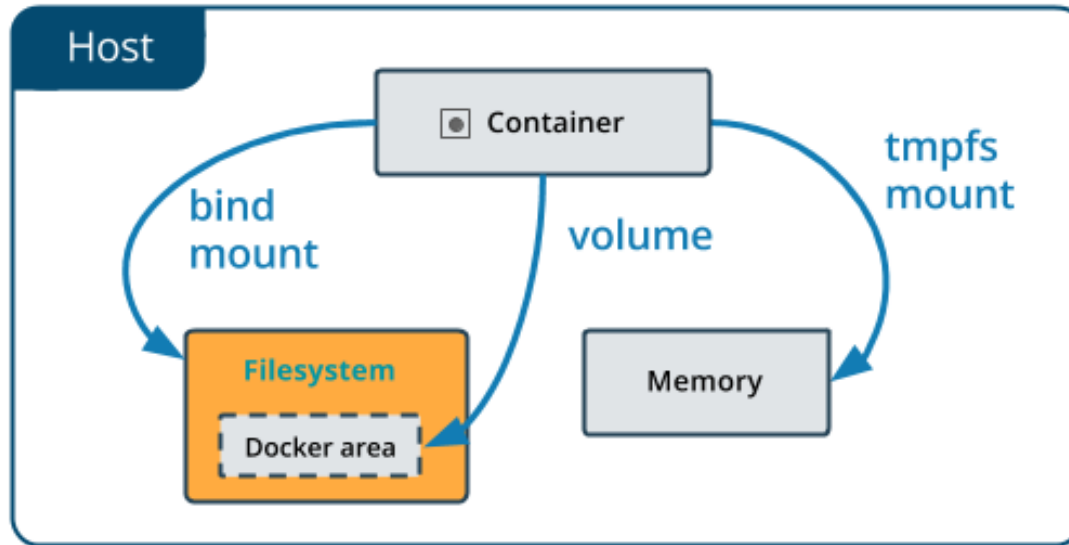
Use docker inspect devtest to verify that the volume was created and mounted correctly

Share data among machines

When building fault-tolerant applications, you might need to configure multiple replicas of the same service to have access to the same files.



Docker bind mount



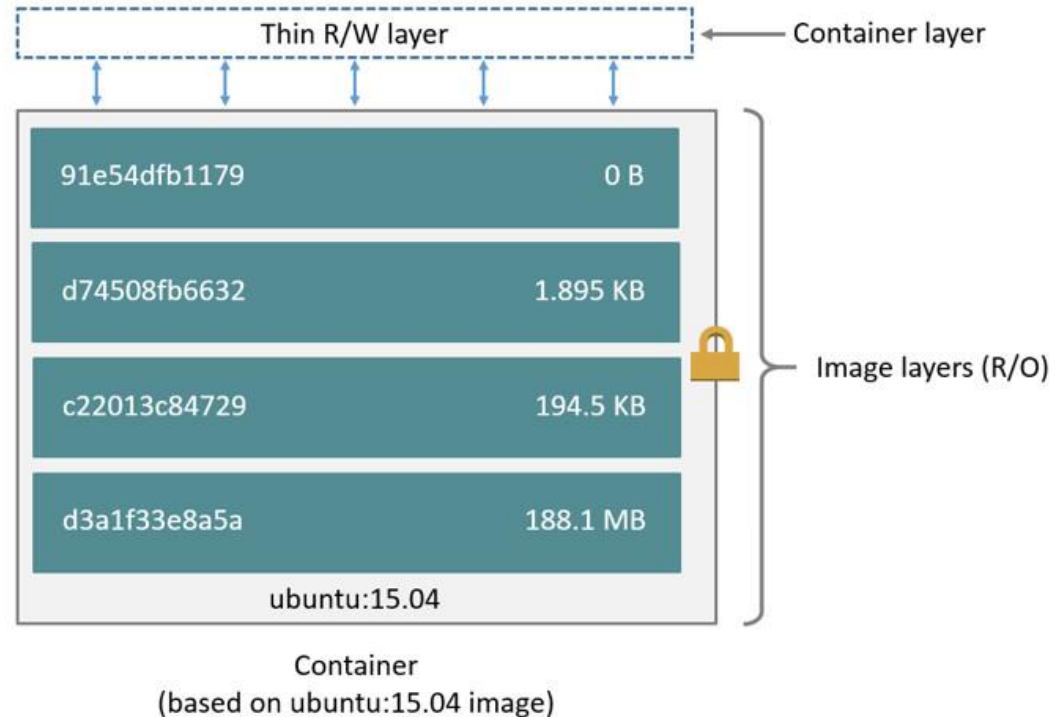
Use tmpfs mounts

Volumes and **bind mounts** let you share files between the host machine and container so that you can persist data even after the container is stopped.

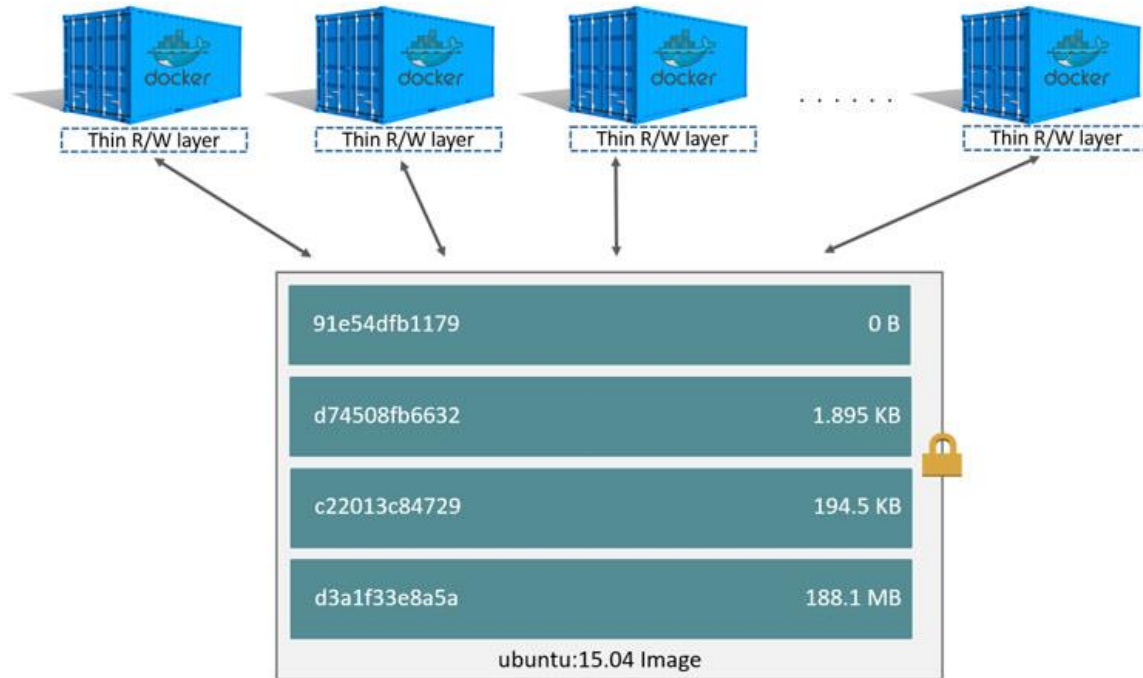
If you're running Docker on Linux, you have a third option: tmpfs mounts. When you create a container with a tmpfs mount, the container can create files outside the container's writable layer.

Storage data within container

A **storage driver** handles the details about the way these layers interact with each other. Different storage drivers are available, which have advantages and disadvantages in different situations.



Container and layers



Docker supports several **storage drivers**, using a pluggable architecture. The **storage driver** controls how images and containers are stored and managed on your Docker host.

After you have read the storage driver overview, the next step is to choose the best storage driver for your workloads. Use the storage driver with the best overall performance and stability in the most usual scenarios.

Docker storage drivers

Driver	Description
<code>overlay2</code>	<code>overlay2</code> is the preferred storage driver for all currently supported Linux distributions, and requires no extra configuration.
<code>fuse-overlayfs</code>	<code>fuse-overlayfs</code> is preferred only for running Rootless Docker on a host that does not provide support for rootless <code>overlay2</code> . On Ubuntu and Debian 10, the <code>fuse-overlayfs</code> driver does not need to be used, and <code>overlay2</code> works even in rootless mode. Refer to the rootless mode documentation for details.
<code>btrfs</code> and <code>zfs</code>	The <code>btrfs</code> and <code>zfs</code> storage drivers allow for advanced options, such as creating “snapshots”, but require more maintenance and setup. Each of these relies on the backing filesystem being configured correctly.
<code>vfs</code>	The <code>vfs</code> storage driver is intended for testing purposes, and for situations where no copy-on-write filesystem can be used. Performance of this storage driver is poor, and is not generally recommended for production use.

`aufs`

The `aufs` storage driver was the preferred storage driver for Docker 18.06 and older, when running on Ubuntu 14.04 on kernel 3.13 which had no support for `overlay2`. However, current versions of Ubuntu and Debian now have support for `overlay2`, which is now the recommended driver.

`devicemapper`

The `devicemapper` storage driver requires `direct-lvm` for production environments, because `loopback-lvm`, while zero-configuration, has very poor performance. `devicemapper` was the recommended storage driver for CentOS and RHEL, as their kernel version did not support `overlay2`. However, current versions of CentOS and RHEL now have support for `overlay2`, which is now the recommended driver.

`overlay`

The legacy `overlay` driver was used for kernels that did not support the “multiple-lowerdir” feature required for `overlay2`. All currently supported Linux distributions now provide support for this, and it is therefore deprecated.

- ❖ Mount local source code into a running container

```
docker container run -v $(pwd):/usr/src/app/ myapp
```

- ❖ Improve performance

As directory structures get complicated traversing the tree can slow system performance

- ❖ Data persistence

PRACTICE DOCKER STORAGE

